

JDBC IN JAVA

05 March 2025 21:40

Full Form: JDBC = Java Database Connectivity:

➤ What is it?

JDBC is a **tool provided by Java** to **connect your Java program to any database** (like MySQL, Oracle, PostgreSQL, etc).

➤ Why is it needed?

Without JDBC, your Java code cannot directly talk to a database. JDBC works like a **bridge** between Java and the Database.

JDBC = Java Database Connectivity

- It is **not a database**.
- It is **not a driver**.
- It is essentially an **API (Application Programming Interface)** in Java.

What that means:

- It's a **set of interfaces and classes** built into Java (in java.sql and javax.sql packages).
- It defines **how Java programs talk to databases**.
- The actual **communication with the database** is done by **JDBC drivers** (provided by database vendors like MySQL, Oracle, PostgreSQL).

Think of JDBC like:

JDBC (API) = the "rules"

JDBC Driver = the "translator"

Database = the "person" you're talking to

Your Java code calls JDBC API methods → driver translates them into DB-specific protocol → database executes them.

➤ JDBC Architecture

There are **four key components**:

Layer	Description
Application	Your Java program
JDBC API	Interfaces and classes (java.sql, javax.sql)
JDBC Driver Manager	Loads the appropriate driver
JDBC Driver	Vendor-specific implementation (MySQL, Oracle, etc.)

➤ JDBC Drivers Types:

There are 4 types of JDBC Drivers:

<u>Type</u>	<u>Name</u>	<u>Description</u>
1	JDBC-ODBC Bridge Driver	Converts JDBC calls to ODBC calls (obsolete)
2	Native API Driver	Uses native client libraries
3	Network Protocol Driver	Middleware server between Java and DB
4	Thin Driver (Pure Java)	Directly talks to DB (most used, e.g. MySQL)

Today, we mainly use **Type 4 (Thin Drivers)**.

What can you do with JDBC?

With JDBC, you can: Connect to Database:

- Run SQL Queries (like SELECT, INSERT, UPDATE, DELETE)
- Fetch data from tables
- Insert new records
- Update existing data
- Delete rows

- **Steps to Use JDBC**

1. Load the Driver

For Ex : there is a java application and there is a Database(SQL) , so if they want to talk each other and you can say that they want to communicate to each other so in this case we want a translator between them , and in the technical term translator is a **Driver**.

2. Create Connection

Now translator will do its job. But now I want to go from java app to Sql. So in this case we have to create a road between them, and in technical term we can say that it is a **Connection**.

3. Create Statement

Now suppose we have some data in our java application , and I want to send this data to the sql so I need any kind of vehicle to deliver my data (Truck) , so in the technical term we can say that it is a **Statement**.

4. Execute Query

So now our data is successfully delivered and stored in our sql database. **So in technical terms we can execute the query.**

5. Process Results (if SELECT)

And in the last it will process the result.

6. Close Everything

===== Steps to connect Java with Database using JDBC =====

1. Load & Register Driver

=> `Class.forName("driver class");`

2. Create connection

=> `Connection con = DriverManager.getConnection("url", "username", "password");`

3. Create statement

=> `PreparedStatement ps = con.prepareStatement("sql query");`

4. Execute SQL statement

=> `ps.executeUpdate();`

5. Process the result

6. Close the connection

At last we have to close the connection.

1 Step 1: Download MySQL Driver Jar

→ Download mysql-connector-j-8.0.33.jar

→ Add it to your project classpath (In Eclipse/IntelliJ)

2 Step 2: Write Full Code

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class JdbcCoreJavaExample {
    public static void main(String[] args) {
        Connection con = null;
        Statement stmt = null;
        ResultSet rs = null;
        try {

// 1. Load MySQL Driver
            Class.forName("com.mysql.cj.jdbc.Driver");
// 2. Create Connection
            String url = "jdbc:mysql://localhost:3306/databaseName";
            String user = "root";
            String password = "password"; // replace with your password
            con = DriverManager.getConnection(url, user, password);
// 3. Create Statement
            stmt = con.createStatement();
// 4. Execute Query
            String sql = "SELECT * FROM students";
            rs = stmt.executeQuery(sql);
// 5. Process ResultSet
            System.out.println("Student List:");
            System.out.println("-----");
            while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                int age = rs.getInt("age");
                System.out.println(id + " - " + name + " - " + age);
            }
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            try {
                // 6. Close everything
                if (rs != null) rs.close();
                if (stmt != null) stmt.close();
                if (con != null) con.close();
            } catch (Exception ex) {
                ex.printStackTrace();
            }
        }
    }
}
```

➤ JDBC Classes & Interfaces

Interface/Class

Purpose

DriverManager

Manages database drivers

Connection

Represents a connection to DB

Statement

Used to execute SQL statements

PreparedStatement

Precompiled SQL with parameters

CallableStatement

Execute stored procedures

ResultSet

Holds the result of SELECT queries

➤ Statement vs PreparedStatement:

Feature	Statement	PreparedStatement
SQL Injection	Vulnerable(open to attack or exploitation.)	Safe
Speed	Slower for repeated queries	Faster for repeated queries
Parameters	Not supported	Supported using ? placeholders

```
PreparedStatement pstmt = con.prepareStatement("INSERT INTO student VALUES (?, ?);");
pstmt.setInt(1, 1);
pstmt.setString(2, "John");
pstmt.executeUpdate();
```

1. Statement:

- **Purpose:** Statement is used for executing simple SQL queries (like SELECT, INSERT, UPDATE, DELETE).
- **SQL Injection Vulnerability:** Statement is susceptible to SQL injection attacks because the SQL query is constructed dynamically by concatenating strings, which allows malicious input to alter the query's behavior.
Example:

```
Statement stmt = conn.createStatement();
String query = "SELECT * FROM users WHERE username = " + username + " AND password = " + password + """,
ResultSet rs = stmt.executeQuery(query);
```

- If username or password contains malicious input, it can compromise the query.
- **Performance:** Statement is less efficient when executing the same query multiple times with different parameters because it does not support query pre-compilation or caching.
- **Use case:** Best used when executing simple queries without any user input or when the query does not need to be run multiple times with different parameters.

➤ What Happens During SQL Injection:

If an attacker provides **malicious input** for username and/or password, they can alter the structure of the SQL query. Since the query is being dynamically built from user input, the attacker can modify the query logic, leading to potentially dangerous consequences.

Example 1: Bypassing Authentication

Let's say the attacker enters the following into the username field:

```
' ' OR '1'='1
```

And anything in the password field (for example, anything). The query becomes:

```
SELECT * FROM users WHERE username = ' ' OR '1'='1' AND password = 'anything';
```

What happens here?

- The OR '1'='1' part of the query is always true. In other words, no matter what value is entered for the password, the condition will always be satisfied.
- As a result, the query might return **all rows from the users table**, effectively bypassing the login mechanism because '1'='1' is always true.

The attacker can now potentially **log in without knowing the actual password** or manipulate the application's behavior.

2. PreparedStatement

- **Purpose:** PreparedStatement is used for executing SQL queries with parameters, and it's often more efficient and secure than Statement.
- **SQL Injection Protection:** PreparedStatement uses placeholders (parameterized queries) to insert user input, which prevents SQL injection because the user input is treated as data, not part of the query. The database can distinguish between query structure and data.

Example:

```
String query = "SELECT * FROM users WHERE username = ? AND password = ?";
PreparedStatement pstmt = conn.prepareStatement(query);
pstmt.setString(1, username); // setting the username parameter
pstmt.setString(2, password); // setting the password parameter
ResultSet rs = pstmt.executeQuery();
```

➤ Transaction Management in JDBC:

By default, JDBC runs in **auto-commit mode** (each statement commits automatically).

You can control transactions manually:

```
try {
    stmt.executeUpdate("UPDATE account SET balance = balance - 500 WHERE id=1");
    stmt.executeUpdate("UPDATE account SET balance = balance + 500 WHERE id=2");
    con.commit(); // commit transaction
} catch (Exception e) {
    con.rollback(); // rollback on error
}
```

➤ Batch Processing:

Execute multiple statements at once for performance:

```
Statement stmt = con.createStatement();
stmt.addBatch("INSERT INTO student VALUES (3, 'A')");
stmt.addBatch("INSERT INTO student VALUES (4, 'B')");
stmt.executeBatch();
```

➤ Handling Exceptions:

Always wrap JDBC code in try-catch and close resources in finally or use try-with-resources:

```
try (Connection con = DriverManager.getConnection(url, user, password);
     Statement stmt = con.createStatement()) {
    // execute queries
} catch (SQLException e) {
    e.printStackTrace();
}
```

➤ ALL IN ONE CODE :

```
import java.sql.*;
import java.util.Arrays;

public class AllInOneJDBCDemo {

    // CHANGE these to match your environment
    private static final String DB_URL = "jdbc:mysql://localhost:3306/demo_db?serverTimezone=UTC";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "password";

    public static void main(String[] args) {
        System.out.println("=== JDBC All-in-One Demo ===");

        // 0. Driver loading (optional for modern drivers if on classpath)
        try {
```

```

    Class.forName("com.mysql.cj.jdbc.Driver");
} catch (ClassNotFoundException e) {
    System.out.println("Driver not found in classpath. Make sure connector jar is added.");
}

// We use a single Connection for the demo. In real apps manage connections with pools.
try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD)) {

    System.out.println("Connected to DB: " + DB_URL);

    // 1. Create a table (DROP if exists to keep demo repeatable)
    createTable(conn);

    // 2. Insert rows using PreparedStatement (safe)
    insertWithPreparedStatement(conn);

    // 3. Insert rows using Statement (simple) and demonstrate batch
    insertWithStatementAndBatch(conn);

    // 4. Read (SELECT) and print rows
    selectAllStudents(conn);

    // 5. Update a row (PreparedStatement)
    updateStudentName(conn, 2, "UpdatedName");

    // 6. Delete a row
    deleteStudent(conn, 3);

    // 7. Alter table: add a column
    addColumnIfNotExists(conn);

    // 8. Transaction demo: transfer simulation with commit & rollback
    transactionDemo(conn);

    // 9. CallableStatement demo (stored procedure)
    storedProcedureDemo(conn);

    // 10. SQL Injection demo: vulnerable Statement vs PreparedStatement
    sqlInjectionDemo(conn);

    // 11. Cleanup: drop table (optional)
    dropTable(conn);

    System.out.println("=== Demo finished ===");
} catch (SQLException e) {
    e.printStackTrace();
}

// 1. Create table
private static void createTable(Connection conn) throws SQLException {
    String drop = "DROP TABLE IF EXISTS student";
    String create = "CREATE TABLE student (" +
        " id INT PRIMARY KEY," +
        " name VARCHAR(100) NOT NULL," +
        " balance DECIMAL(10,2) DEFAULT 0.00" +
        ")";
    try (Statement stmt = conn.createStatement()) {
        stmt.executeUpdate(drop);
        stmt.executeUpdate(create);
        System.out.println("[CREATE] Table 'student' created.");
    }
}

```

```

// 2. Insert using PreparedStatement (parameterized -> safe)
private static void insertWithPreparedStatement(Connection conn) throws SQLException {
    String insertSql = "INSERT INTO student (id, name, balance) VALUES (?, ?, ?)";
    try (PreparedStatement ps = conn.prepareStatement(insertSql)) {
        // row 1
        ps.setInt(1, 1);
        ps.setString(2, "Alice");
        ps.setBigDecimal(3, new java.math.BigDecimal("1000.00"));
        ps.executeUpdate();

        // row 2
        ps.setInt(1, 2);
        ps.setString(2, "Bob");
        ps.setBigDecimal(3, new java.math.BigDecimal("500.00"));
        ps.executeUpdate();

        System.out.println("[INSERT] 2 rows inserted with PreparedStatement (safe).");
    }
}

// 3. Insert using Statement & batch (for demonstration)
private static void insertWithStatementAndBatch(Connection conn) throws SQLException {
    try (Statement stmt = conn.createStatement()) {
        // row 3
        stmt.addBatch("INSERT INTO student (id, name, balance) VALUES (3, 'Charlie', 300.00)");
        // row 4
        stmt.addBatch("INSERT INTO student (id, name, balance) VALUES (4, 'David', 200.00)");

        int[] results = stmt.executeBatch(); // run both as batch
        System.out.println("[BATCH INSERT] executed, batch sizes/results: " + Arrays.toString(results));
    }
}

// 4. Read all rows
private static void selectAllStudents(Connection conn) throws SQLException {
    String q = "SELECT id, name, balance FROM student ORDER BY id";
    try (Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(q)) {
        System.out.println("[SELECT] Current rows in student:");
        while (rs.next()) {
            int id = rs.getInt("id");
            String name = rs.getString("name");
            java.math.BigDecimal bal = rs.getBigDecimal("balance");
            System.out.printf(" id=%d, name=%s, balance=%s%n", id, name, bal);
        }
    }
}

// 5. Update a student's name
private static void updateStudentName(Connection conn, int id, String newName) throws SQLException {
    String updateSql = "UPDATE student SET name = ? WHERE id = ?";
    try (PreparedStatement ps = conn.prepareStatement(updateSql)) {
        ps.setString(1, newName);
        ps.setInt(2, id);
        int rows = ps.executeUpdate();
        System.out.println("[UPDATE] Rows updated: " + rows + " (set id=" + id + " name=" + newName + ")");
    }
}

// 6. Delete a student
private static void deleteStudent(Connection conn, int id) throws SQLException {
    String delSql = "DELETE FROM student WHERE id = ?";
    try (PreparedStatement ps = conn.prepareStatement(delSql)) {

```

```

        ps.setInt(1, id);
        int rows = ps.executeUpdate();
        System.out.println("[DELETE] Rows deleted: " + rows + " (id=" + id + ")");
    }
}

// 7. Alter table: add column if not exists (MySQL-specific check)
private static void addColumnIfNotExists(Connection conn) throws SQLException {
    // Simple approach: attempt ALTER and ignore exception if column exists.
    String alterSql = "ALTER TABLE student ADD COLUMN IF NOT EXISTS email VARCHAR(100) DEFAULT NULL";
    // Note: "IF NOT EXISTS" for ADD COLUMN is supported in newer MySQL versions. If your MySQL doesn't
    // support it,
    // you can check information_schema.columns first.
    try (Statement stmt = conn.createStatement()) {
        stmt.executeUpdate(alterSql);
        System.out.println("[ALTER] Column 'email' ensured (added if missing).");
    } catch (SQLException e) {
        // If DB doesn't support 'IF NOT EXISTS', fallback:
        if (e.getMessage().toLowerCase().contains("syntax")) {
            System.out.println("[ALTER] Server doesn't support ADD COLUMN IF NOT EXISTS; skipping
automatic add.");
        } else {
            throw e;
        }
    }
}

// 8. Transaction demo (transfer funds between two students)
private static void transactionDemo(Connection conn) throws SQLException {
    System.out.println("[TRANSACTION] Starting transaction demo (transfer 100 from id=1 to id=2)");
    String withdrawSql = "UPDATE student SET balance = balance - ? WHERE id = ?";
    String depositSql = "UPDATE student SET balance = balance + ? WHERE id = ?";

    // Save auto-commit state to restore later
    boolean oldAutoCommit = conn.getAutoCommit();
    conn.setAutoCommit(false); // start transaction

    try (PreparedStatement withdraw = conn.prepareStatement(withdrawSql);
        PreparedStatement deposit = conn.prepareStatement(depositSql)) {

        // withdraw 100 from id=1
        withdraw.setBigDecimal(1, new java.math.BigDecimal("100.00"));
        withdraw.setInt(2, 1);
        withdraw.executeUpdate();

        // deposit 100 to id=2
        deposit.setBigDecimal(1, new java.math.BigDecimal("100.00"));
        deposit.setInt(2, 2);
        deposit.executeUpdate();

        // commit transaction
        conn.commit();
        System.out.println("[TRANSACTION] Commit successful.");
    } catch (SQLException e) {
        // rollback on error
        conn.rollback();
        System.out.println("[TRANSACTION] Error occurred; rolled back transaction. Error: " + e.getMessage());
    } finally {
        // restore original auto-commit
        conn.setAutoCommit(oldAutoCommit);
    }
}

```

```

    // show balances after transaction attempt
    selectAllStudents(conn);
}

// 9. Stored procedure / CallableStatement demo
private static void storedProcedureDemo(Connection conn) throws SQLException {
    System.out.println("[CALLABLE] Stored procedure demo (create & call) - if allowed by DB user.");
    // NOTE: Creating stored procedures may require special privileges. If not allowed, run the CREATE
    PROCEDURE SQL
    // manually in your DB, then only run the CALL part below.

    String createProc = ""
        + "DROP PROCEDURE IF EXISTS get_student_name; "
        + "CREATE PROCEDURE get_student_name(IN in_id INT)\n"
        + "BEGIN\n"
        + " SELECT name FROM student WHERE id = in_id;\n"
        + "END;";

    // Some JDBC drivers don't accept multiple statements in one executeUpdate by default.
    // To keep this robust, try to create the proc using one statement at a time.
    try (Statement st = conn.createStatement()) {
        // Try dropping and creating with separate statements:
        st.executeUpdate("DROP PROCEDURE IF EXISTS get_student_name");
        st.executeUpdate("CREATE PROCEDURE get_student_name(IN in_id INT) BEGIN SELECT name
FROM student WHERE id = in_id; END");
        System.out.println("[CALLABLE] Stored procedure 'get_student_name' created.");
    } catch (SQLException e) {
        System.out.println("[CALLABLE] Could not create stored procedure automatically. Message: " +
e.getMessage());
        System.out.println("[CALLABLE] If creation failed, create procedure manually and the CALL below will
still work (if proc exists).");
    }

    // Now call the procedure
    try (CallableStatement cs = conn.prepareCall("{CALL get_student_name(?)}")) {
        cs.setInt(1, 1);
        boolean hasResultSet = cs.execute();

        if (hasResultSet) {
            try (ResultSet rs = cs.getResultSet()) {
                System.out.println("[CALLABLE] Procedure result:");
                while (rs.next()) {
                    System.out.println(" name = " + rs.getString(1));
                }
            }
        } else {
            System.out.println("[CALLABLE] Procedure returned no result set.");
        }
    } catch (SQLException e) {
        System.out.println("[CALLABLE] Error calling stored procedure: " + e.getMessage());
    }
}

// 10. SQL injection demo: show vulnerable concatenation vs PreparedStatement
private static void sqlInjectionDemo(Connection conn) throws SQLException {
    System.out.println("[SQL INJECTION DEMO] Demonstrating vulnerable Statement vs
PreparedStatement.");

    // Setup a simple 'users' table for login demo
    try (Statement st = conn.createStatement()) {
        st.executeUpdate("DROP TABLE IF EXISTS users_demo");
        st.executeUpdate("CREATE TABLE users_demo (username VARCHAR(50) PRIMARY KEY, password
VARCHAR(50))");
    }
}

```

```

        st.executeUpdate("INSERT INTO users_demo (username, password) VALUES ('admin', 'adminpass')");
        st.executeUpdate("INSERT INTO users_demo (username, password) VALUES ('user', 'userpass')");
    }

    // Simulated attacker input
    String attackerUsernamePayload = " OR '1'='1' -- ";
    String attackerPassword = "any"; // attacker knows password is irrelevant for this payload

    // 10a. Vulnerable approach (DO NOT use in production)
    String vulnerableSql = "SELECT * FROM users_demo WHERE username = " + attackerUsernamePayload
        + " AND password = " + attackerPassword + "";
    System.out.println("VULNERABLE SQL (constructed): " + vulnerableSql);

    try (Statement s = conn.createStatement();
        ResultSet rs = s.executeQuery(vulnerableSql)) {
        int count = 0;
        while (rs.next()) count++;
        System.out.println("VULNERABLE QUERY returned rows: " + count + " (attacker bypass successful
if >0)");
    } catch (SQLException e) {
        System.out.println("[VULNERABLE] Error executing vulnerable SQL: " + e.getMessage());
    }

    // 10b. Safe approach: PreparedStatement
    String safeSql = "SELECT * FROM users_demo WHERE username = ? AND password = ?";
    try (PreparedStatement ps = conn.prepareStatement(safeSql)) {
        ps.setString(1, attackerUsernamePayload);
        ps.setString(2, attackerPassword);
        try (ResultSet rs = ps.executeQuery()) {
            int count = 0;
            while (rs.next()) count++;
            System.out.println("PREPARED STATEMENT returned rows: " + count + " (should be 0 for attack
payload)");
        }
    }
    System.out.println("[SQL INJECTION DEMO] Finished. Showed how prepared statements prevent payloads
from altering SQL logic.");
}

// 11. Drop table (cleanup)
private static void dropTable(Connection conn) throws SQLException {
    try (Statement stmt = conn.createStatement()) {
        stmt.executeUpdate("DROP TABLE IF EXISTS student");
        stmt.executeUpdate("DROP TABLE IF EXISTS users_demo");
        stmt.executeUpdate("DROP PROCEDURE IF EXISTS get_student_name");
        System.out.println("[CLEANUP] Dropped demo objects (student, users_demo, get_student_name).");
    }
}
}

```